

Camera and Track Following with the TurtleBot3

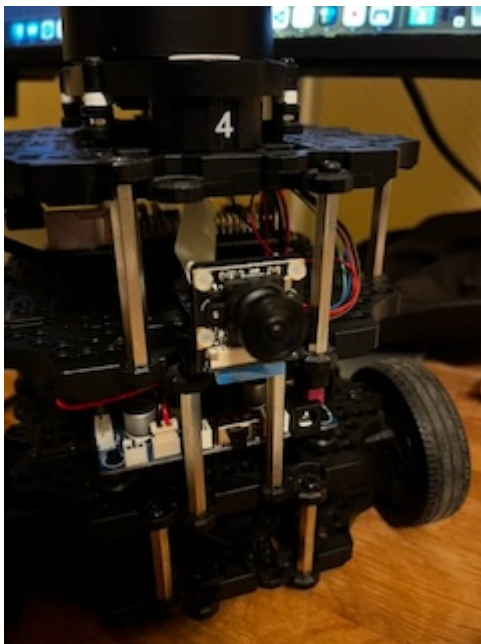
(Version française ici)

This tutorial shows how to make the TurtleBot3 follow a colored line, then a track made of two colored lines, using its camera. It offers a solution intended to be simpler and more accessible than the track following of the [autonomous_driving](#) project in the official manual.

It assumes that:

- a working TurtleBot3 is available, with a camera mounted at the front of the robot and connected to the Raspberry Pi (RPi): a [RPi fisheye camera \(M\)](#), whose wide field of view is an advantage, or a [RPi camera v2](#),
- both the RPi SD card of the TurtleBot3 and the PC have been set up with ROS 2 Humble,
- the RPi SD card is configured to use the legacy camera driver and the [v4l2_camera](#) ROS package is installed, either:
 - because you are using the ready-to-use SD card image, which already includes them ([Quick_installation](#)),
 - or because you have set up the RPi yourself as described in the manual: [quick_start_guide/sbc_setup#raspberry-pi-camera](#).

All the Python scripts used in this tutorial are in the [src/](#) folder.



A fisheye camera mounted at the front of TB3

Contents

1. [Camera test](#)
2. [A line follower](#)
3. [A track follower \(two lines\)](#)
4. [Appendix: embed the Python script in a ROS 2 package](#)

1. Camera test ([v4l2_camera](#) ROS package)

Most of the camera test is described in the official manual ([quick_start_guide/sbc_setup#raspberry-pi-camera](#)). It is repeated below.

On the TurtleBot3 (RPI, through SSH)

1. Check that the camera is detected:

```
v4l2-ctl --list-devices
```

It should display:

```
mmal service 16.1 (platform:bcm2835-v4l2-0):  
  /dev/video0
```

Note: `v4l2-ctl --all` displays all the camera parameters that can be modified (auto-exposure, frame rate, ...). For example, `v4l2-ctl -p 10` sets the frame rate to 10 fps instead of the default (30 fps or more).

2. Run the camera node, which streams both raw and compressed images:

```
ros2 run v4l2_camera v4l2_camera_node
```

It should display:

```
[INFO] [1790633826.244848025] [v4l2_camera]: Success  
[INFO] [1790633826.247136817] [v4l2_camera]: Starting camera  
[WARN] [1790633826.747098130] [v4l2_camera]: Image encoding not the  
same as requested output, performing possibly slow conversion:  
yuv422_yuy2 => rgb8  
[INFO] [1790633826.834411816] [v4l2_camera]: using default calibration  
URL  
[INFO] [1790633826.834613166] [v4l2_camera]: camera calibration URL:  
file:///home/ubuntu/.ros/camera_info/mmal_service_16.1.yaml  
[ERROR] [1790633826.834948385] [camera_calibration_parsers]: Unable to  
open camera calibration file  
[/home/ubuntu/.ros/camera_info/mmal_service_16.1.yaml]  
[WARN] [1790633826.835030513] [v4l2_camera]: Camera calibration file  
/home/ubuntu/.ros/camera_info/mmal_service_16.1.yaml not found
```

The important lines are **Success** and **Starting camera**. The **ERROR** about the missing calibration file can be ignored: no camera calibration is needed in this tutorial.

On the PC

1. Make sure the packages needed to decompress the video stream (and to convert ROS images to OpenCV) are installed:

```
sudo apt install ros-humble-image-transport-plugins ros-humble-cv-bridge python3-opencv
```

2. With `v4l2_camera_node` running on the TurtleBot3, check that the camera topics are available:

```
ros2 topic list
```

```
/camera_info  
/image_raw  
/image_raw/compressed  
/image_raw/compressedDepth  
/image_raw/theora  
/parameter_events  
/rosout
```

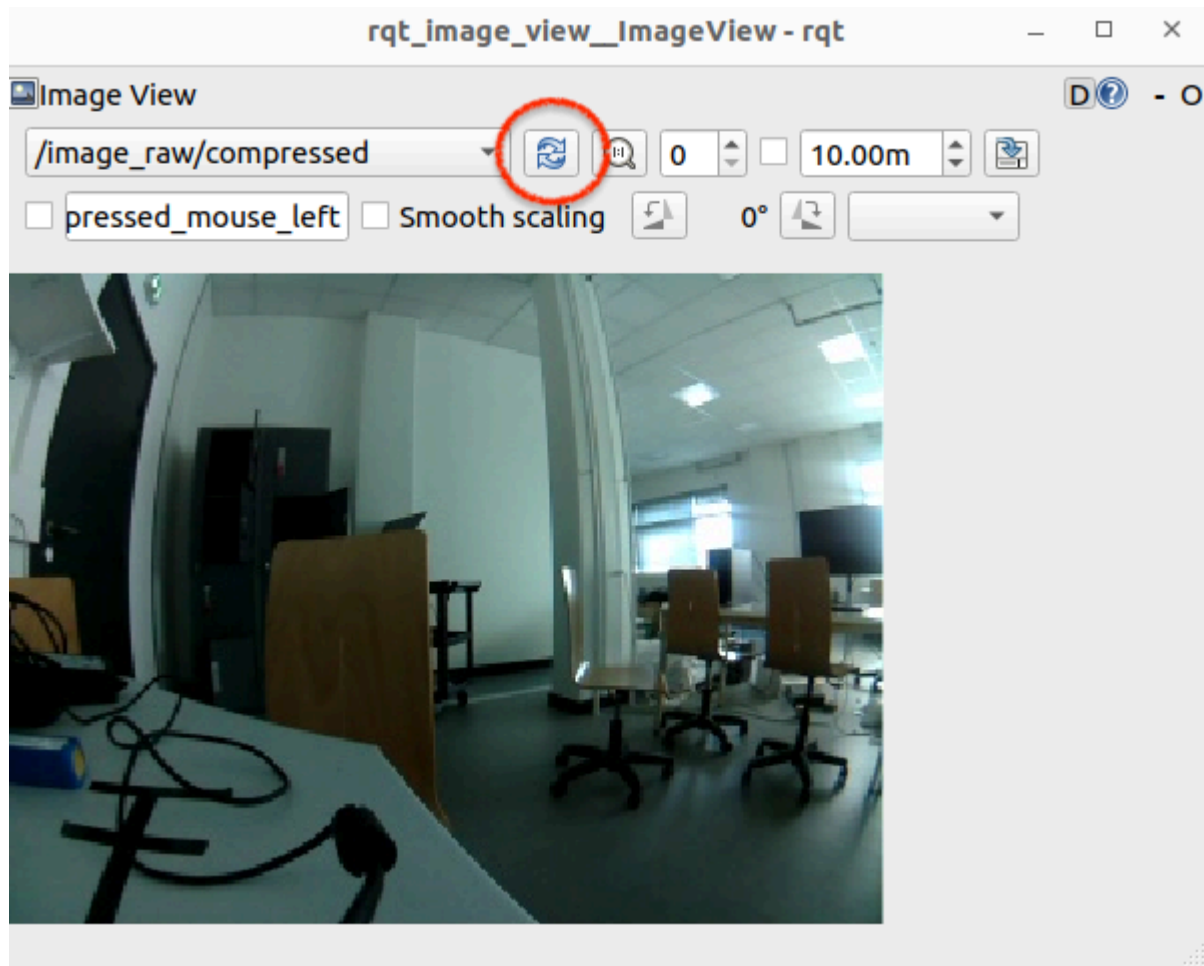
The topics `/image_raw` and `/image_raw/compressed` carry the raw and the compressed video stream respectively.

3. Display the image with the `rqt` graphical tool:

```
rqt -s image_view
```

(or equivalently `ros2 run rqt_image_view rqt_image_view`)

In the window, select the topic `/image_raw` or `/image_raw/compressed`. If a topic is missing from the list, click the refresh button.



Important: the display lags when uncompressed images are streamed over Wi-Fi. **Always use the compressed video stream during missions, so that the Wi-Fi bandwidth is not saturated and the video is not delayed.**

To reduce the bandwidth even more, as recommended in the official manual, lower the image resolution by starting the camera node on the TurtleBot3 with an extra parameter:

```
ros2 run v4l2_camera v4l2_camera_node --ros-args -p image_size:=[320,240]
```

where `[320,240]` is the image resolution (width, height) in pixels.

2. A line follower

Principle

Here, the track to follow is a single colored line (yellow) on the ground.

The robot moves forward at a **constant linear speed**. Its **angular speed** depends on the position of the line in the image of the camera mounted on the robot:

- if the line is in the center of the image, the distance between the line and the image center is zero, so the angular speed is zero and the robot goes straight;

- in a turn, the line is no longer in the center of the image, so the robot turns with a non-zero angular speed, proportional to this distance (proportional control).

Block diagram and processing pipeline

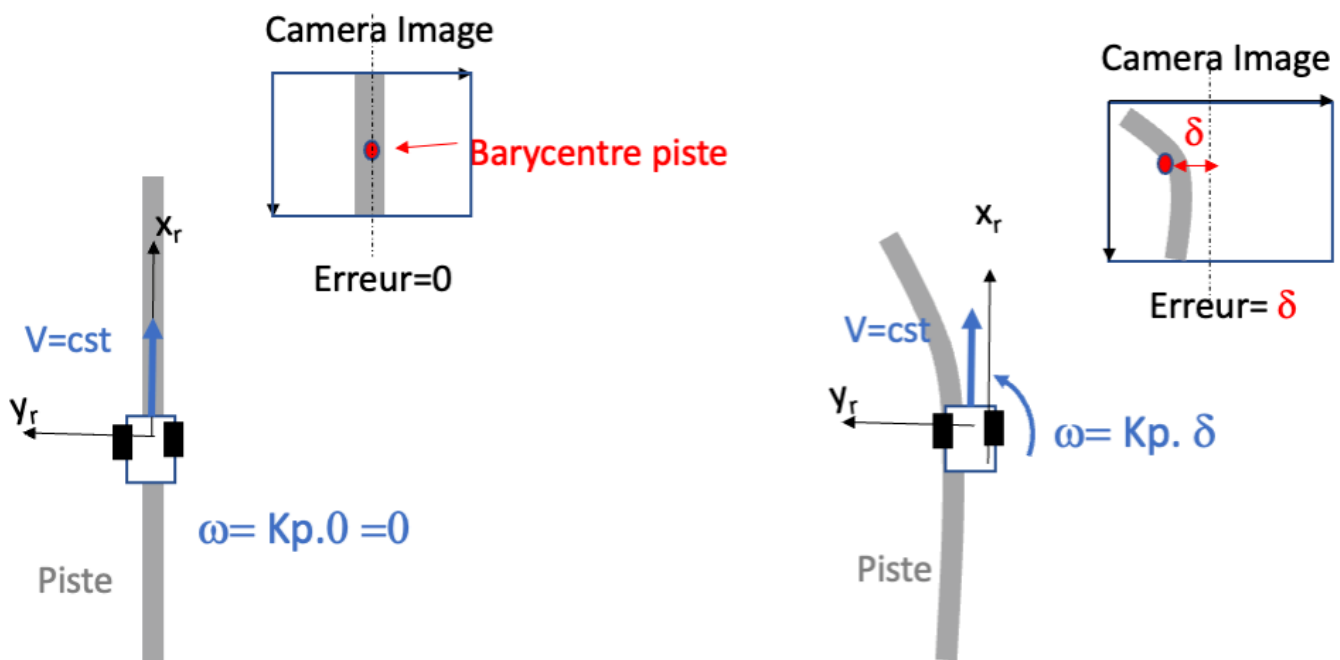
The general control scheme is as follows:

- The Raspberry Pi (RPI) sends a compressed video stream from its camera to the PC.
- The PC:
 - decompresses the video stream,
 - keeps only the bottom part of the frame (the floor just in front of the robot),
 - detects the center (centroid) of the yellow line using color thresholding,
 - computes the horizontal distance between the center of the image and the line, which gives an error δ ,
 - converts this error into an angular speed command ω ,
 - sends this command to the TurtleBot3.
- The TurtleBot3 (RPI) receives this angular speed setpoint and recenters its trajectory on the line, while keeping a constant linear speed V .

Proportional control

The principle of this proportional control is illustrated in the figure below:

- Left: the nominal situation, where the robot moves straight ahead. With no error, the angular speed ω is zero.
- Right: a position error δ along the horizontal axis is measured between the center of the image and the centroid of the line pixels (red dot). This pixel error is converted into an angular speed $\omega = K_p \cdot \delta$ that compensates for it.



Step-by-step implementation

Each step below adds a feature to the script of the previous step. In the code, most of the new lines are preceded by a comment explaining them.

For every step, the camera node must be running **on the TurtleBot3 RPi** (see [section 1](#)), and the script is run **on the PC**, from the [src/](#) folder.

Every script opens OpenCV windows. To stop a script:

- cleanly, press **q** or **Esc** after clicking on an OpenCV window to give it focus,
- or press **Ctrl+C** in the terminal (keyboard interrupt).

Step 1. Image display

Here, we just get the camera stream on the PC and display it with the image processing library **OpenCV**.

- Open [src/01_follower_opencv.py](#) (for example with VSCodium and its Python extension, for syntax highlighting and debugging).
- Read this short script. It:
 - defines an **ImageViewer** class that inherits from **Node** (a ROS node);
 - at object creation (**__init__** method):
 - creates a **quit_requested** variable, set to **False**,
 - subscribes (**create_subscription()**) to the topic where the compressed camera video is streamed (**'/image_raw/compressed'**). Each time a new frame arrives, the class method **image_callback** is called;
 - in the **image_callback(self, msg)** method, called for each new video frame:
 - decompresses the frame and converts it into an OpenCV image (**self.bridge.compressed_imgmsg_to_cv2()**),
 - creates an OpenCV window and gives it the image to display: **cv2.imshow()**,
 - actually displays (refreshes) the image on screen and checks for a key press: **cv2.waitKey()**,
 - if the **q** or **Esc** key is pressed, sets **quit_requested** to **True**;
 - the **main()** function, called when the script is run directly by Python:
 - creates an **ImageViewer** node,
 - as long as **quit_requested** is **False**, lets the node process a new image or any other pending event, once (**rclpy.spin_once(node, timeout_sec=0.1)**). If there is nothing to process, the call returns when the timeout expires,
 - when the user presses **Ctrl+C**, or when **quit_requested** becomes **True**, the **finally** block is executed. It:
 - destroys the OpenCV window(s),
 - destroys the node and frees its resources (**node.destroy_node()**),
 - checks whether the ROS 2 Python context is still running and, if so, shuts it down (**rclpy.shutdown()**).
- Test this node:
 1. If required, replace the compressed video topic **'/image_raw/compressed'** in the code with the topic where your compressed stream is published. The topic can be checked with **ros2 topic list** or by displaying the video with **rqt -s image_view**.

2. Connect to the TurtleBot3 with SSH and start the camera stream, with a reduced resolution of 320x240 to cope with the limited Wi-Fi bandwidth:

```
ros2 run v4l2_camera v4l2_camera_node --ros-args -p image_size:=  
[320,240]
```

The output may look like:

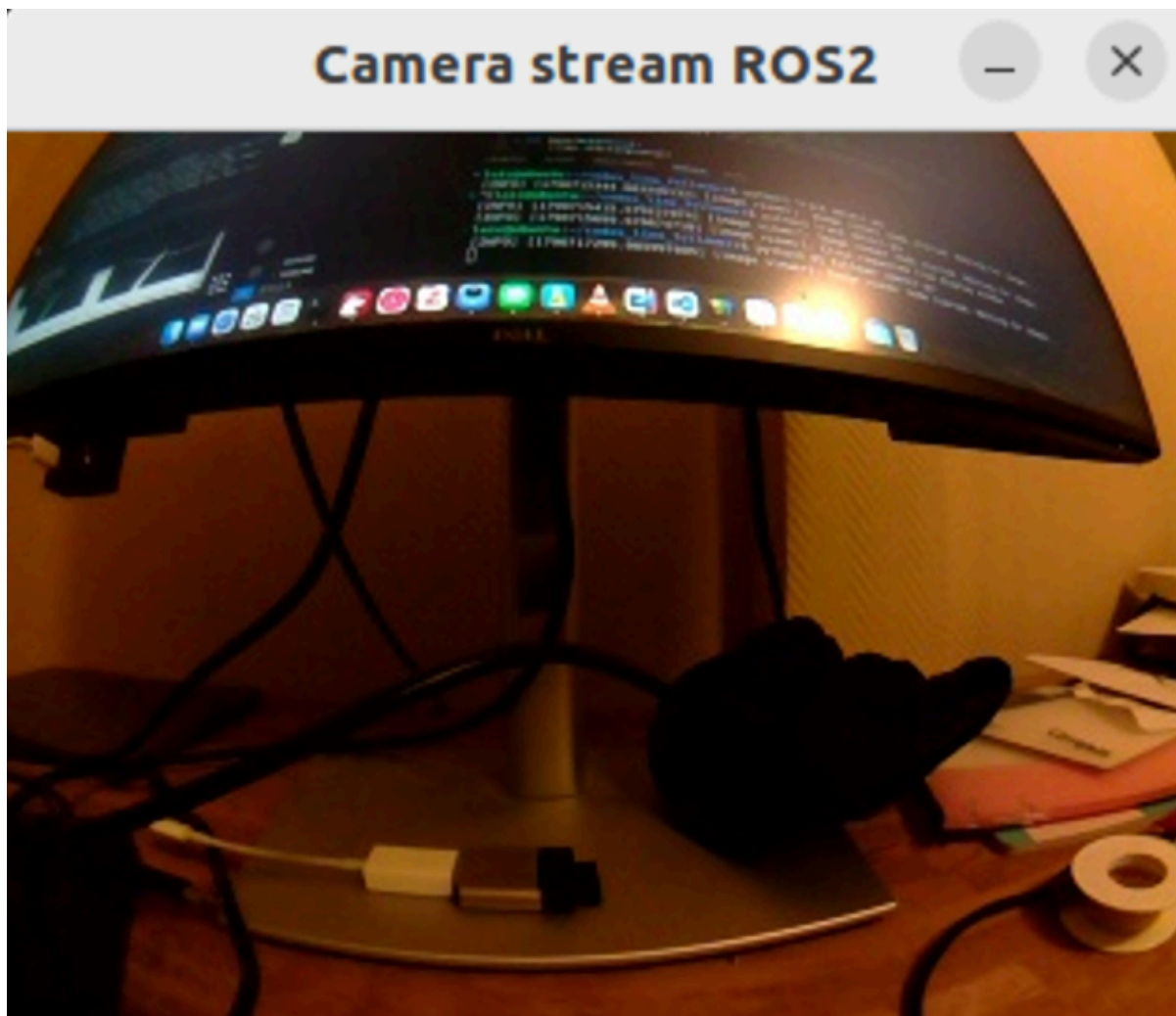
```
[INFO] [1790716607.586749823] [v4l2_camera]: Driver: bm2835 mmal  
[INFO] [1790716607.587212781] [v4l2_camera]: Version: 331693  
...  
[INFO] [1790716607.596011352] [v4l2_camera]: Starting camera  
[WARN] [1790716608.038340088] [v4l2_camera]: Image encoding not  
the same as requested output, performing possibly slow  
conversion: yuv422_yuy2 => rgb8  
[INFO] [1790716608.044575756] [v4l2_camera]: using default  
calibration URL  
[INFO] [1790716608.044843957] [v4l2_camera]: camera calibration  
URL: file:///home/ubuntu/.ros/camera_info/mmal_service_16.1.yaml  
[ERROR] [1790716608.045181306] [camera_calibration_parsers]:  
Unable to open camera calibration file  
[/home/ubuntu/.ros/camera_info/mmal_service_16.1.yaml]  
[WARN] [1790716608.045303360] [v4l2_camera]: Camera calibration  
file /home/ubuntu/.ros/camera_info/mmal_service_16.1.yaml not  
found
```

You can ignore the **ERROR** about the missing calibration file.

3. On the PC, run the script:

```
python3 01_follower_opencv.py
```

A window displaying the camera video should open on the PC:



Check that the displayed video has no latency. If it lags because of the Wi-Fi connection, lower the resolution (if not done already) or the frame rate.

4. Stop the script with `q`, `Esc` or `Ctrl+C`.

- Troubleshooting (if no image is displayed):
 - check the active nodes with `rqt_graph` or `ros2 node list`, and verify that the `v4l2_camera` node is present;
 - if the camera node is not visible from the PC, check that the PC and the TurtleBot3 are on the same network and use the same `ROS_DOMAIN_ID` (`echo $ROS_DOMAIN_ID` on both);
 - run `rqt -s image_view` to check that a video stream is received;
 - check that the topic in `01_follower_opencv.py` (`'/image_raw/compressed'`) matches the topic where the compressed stream is displayed in `rqt`;
 - check that OpenCV, `cv_bridge` and the image transport plugins are installed (`sudo apt install ros-humble-image-transport-plugins ros-humble-cv-bridge python3-opencv`).

Step 2. Color line detection

Starting from the previous script, we now add the detection of the line color.

The color is defined as a range in the HSV (Hue, Saturation, Value) color space:

HSV (Hue, Saturation and Value)

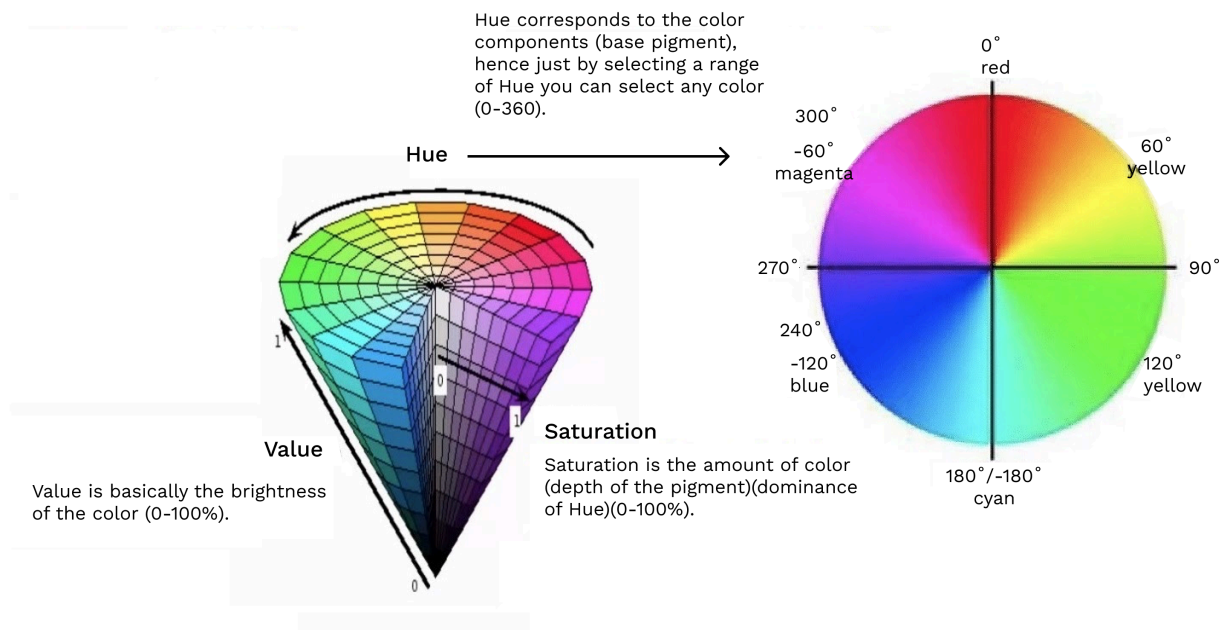


Image source: [Xiao X, Li J, Zhou M, Gao H, Wang Q, Xia Y, Lim J and Xu Z \(2026\) Carotid vulnerable plaque in coronary heart disease: a machine learning-based diagnostic model integrating tongue parameters and blood metabolic biomarkers. Front. Cardiovasc. Med.](#)

In OpenCV, saturation S and value V are in [0, 255] (i.e. 0-100 %) and hue H is in [0, 179] instead of [0°, 360°]. So yellow (60° in the picture above) is $H = 60 / 2 = 30$ in OpenCV.

To detect a yellow line, whose shade varies with light reflections, we select the colors between the lower limit `[20, 100, 100]` and the upper limit `[35, 255, 255]`. If the background is white, light yellows close to white can be excluded by raising the lower limits of saturation and value.

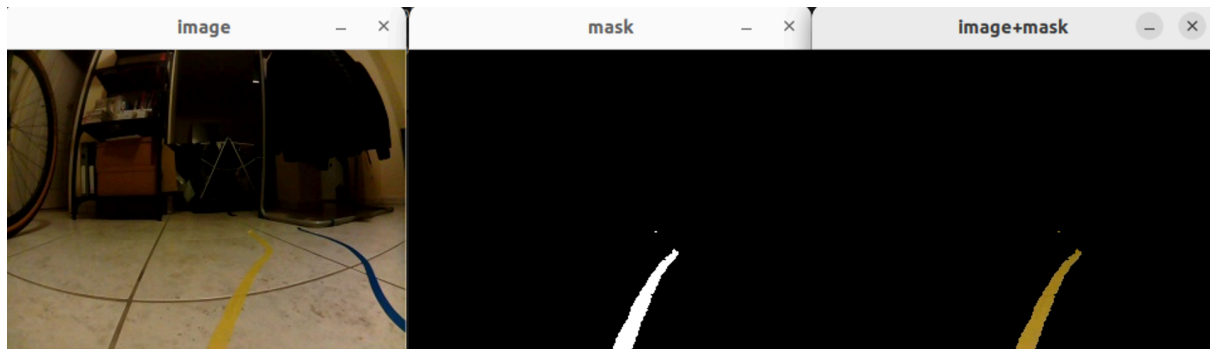
- Open [src/02follower_color_filter.py](#), an extended version of the previous script.
- What is new compared to the previous script:
 - in `__init__`, the lower and upper color limits of a yellow line are defined (`self.lower_color, self.upper_color`);
 - in the `image_callback` method:
 - each frame is converted to the HSV color space (`cv2.cvtColor()`),
 - a binary image called `mask` is created (`cv2.inRange()`): the pixels of the frame (`cv_image`) whose color is within the limits are set to white, all the others to black,
 - for debugging (or for fun), an image called `masked` is created (`cv2.bitwise_and()`): it shows the original pixels where the mask is white, and black elsewhere,
 - finally, the 3 images are displayed.
- Test the script:

1. As before, start the `v4l2_camera` node on the TurtleBot3 RPi.

2. **On the PC**, run the script (after updating the compressed image topic if required):

```
python3 02follower_color_filter.py
```

It should display the original image, the **mask** image (where the color is detected) and the combined image **masked**:



3. Stop the script with **q**, **Esc** or **Ctrl+C**.

4. Adjust the **self.lower_color** and **self.upper_color** values to improve the segmentation of the yellow line while rejecting the background pixels. Run the script again.

Keep in mind that any change to the color limits must also be copied into the scripts of the next steps.

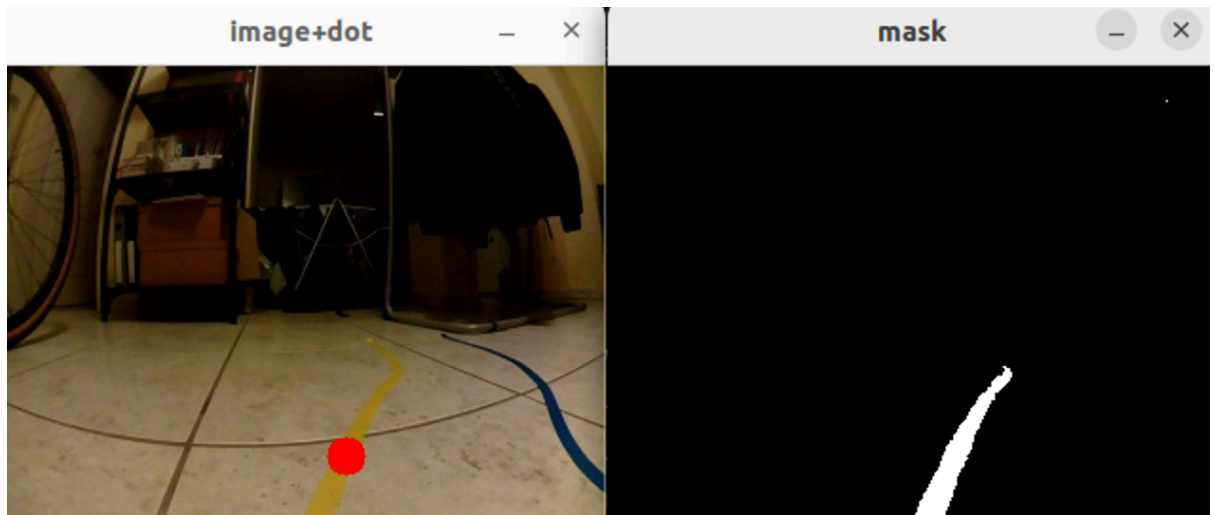
Step 3. Centroid of the colored line

Here, the centroid of the pixels matching the line color is computed and displayed on the image as a red dot. The centroid is the mean position (geometric center) of all the pixels matching the line color. It gives the lateral position of the line in the image.

- Open [src/03follower_center_finder.py](#), an extended version of the previous script.
- What is new compared to the previous script, in the **image_callback** method:
 - the image moments of the mask are computed (**cv2.moments()**),
 - if at least one pixel matches the color (**M['m00'] > 0**), the centroid coordinates are computed: **cx = m10 / m00**, **cy = m01 / m00**,
 - a red circle (**(0, 0, 255)** in BGR) is drawn at the centroid position on the original image: **cv2.circle()**.
- Test the script:
 1. As before, make sure the **v4l2_camera** node is running **on the TurtleBot3 RPi**.
 2. **On the PC**, run the script (after updating the compressed image topic if required):

```
python3 03follower_center_finder.py
```

It should display the original image **with a red dot at the centroid of the yellow pixels**, and the **mask** image (where the color is detected):



3. Stop the script with **q**, **Esc** or **Ctrl+C**.

Note: if several blobs of the same color are visible, the centroid is the center of all of them combined, which may lie between them.

Step 4. Current line position: the bottom ROI

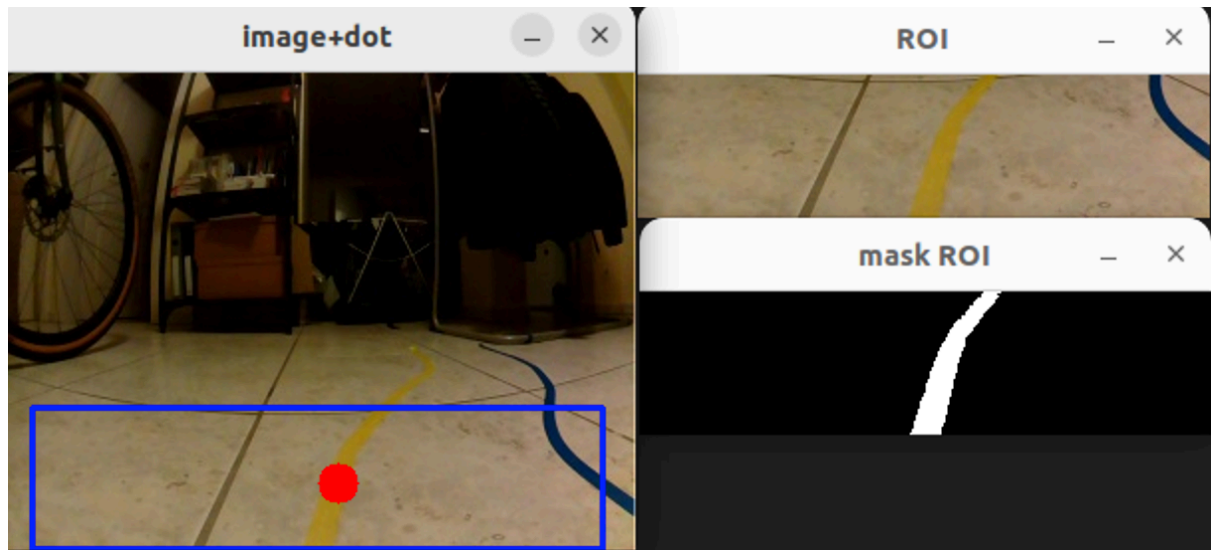
Here, we compute the position of the line **with respect to the robot**. The current position of the line is given only by the part of the line close to the robot, not by the part far ahead. That is, the position of the line in the bottom part of the image. This bottom part is our ROI (Region Of Interest).

So the centroid of the line pixels is now computed only inside the bottom ROI of the image.

- Open [src/04follower_line_finder.py](#), an extended version of the previous script.
- What is new compared to the previous script, in the `image_callback` method:
 - an ROI image is created by copying a part of the original image: `cv_image[top_y:bottom_y, left_x:right_x].copy()`. Adjust `top_y`, `bottom_y`, `left_x` and `right_x` to select the corners of the desired `image_roi` (by default: the bottom 30 % of the image, without the 5 % left and right borders);
 - the ROI boundaries are drawn in blue on the original image;
 - the centroid of the color-matching pixels is now computed in the ROI only, discarding all the pixels outside it (far from the robot). It is then converted back to full-image coordinates by adding the ROI offset (`left_x`, `top_y`).
- Test the script:
 1. As before, make sure the `v4l2_camera` node is running **on the TurtleBot3 RPi**.
 2. **On the PC**, run the script (after updating the compressed image topic if required):

```
python3 04follower_line_finder.py
```

It should display the original image with a red dot at the centroid of **the yellow pixels inside the ROI**, the ROI image and the ROI mask (the ROI pixels where the color is detected):



3. Stop the script with **q**, **Esc** or **Ctrl+C**.

Step 5. Follow the line

Now that the position of the line with respect to the robot is known, the robot can follow it. As explained above, the robot moves at a constant linear velocity and its angular (rotation) velocity keeps the line in the center of the image. In other words, the angular velocity is adjusted so that the red dot stays in the middle of the image.

- Open [src/05follower.py](#), an extended version of the previous script.
- What is new compared to the previous script:
 - at the top of the file, two global variables (used as constants):
 - **LINEAR_VEL**: the constant linear (forward) velocity of the robot, in m/s,
 - **KP**: the proportional gain between the line position error in the image (in pixels) and the angular (turning) velocity, in rad/s;
 - in the `__init__` method:
 - a publisher is created (`self.create_publisher()`) to send a pair of linear and angular velocities (a **Twist** message) to the TurtleBot3 on the **cmd_vel** topic;
 - in the `image_callback` method:
 - if at least one color-matching pixel is detected:
 - the position of the line relative to the robot, i.e. relative to the center of the image, is computed: `err = width / 2 - cx` (`err > 0`: line on the left; `err < 0`: line on the right),
 - the angular velocity is `KP * err`: the farther the robot is from the line, the faster it turns back towards it,
 - the (constant) linear velocity and the angular velocity are published to the robot as a **Twist** (`self.cmd_vel_pub.publish()`);
 - if no line pixel is detected:
 - a zero-velocity **Twist()** is sent to stop the robot;
 - a `stop_robot()` method is added, which sends a zero velocity to the robot;

- in `main()`:
 - the `finally` block (always executed) now calls `stop_robot()` to send a last zero velocity to the robot. Otherwise, the robot would keep moving with the last velocity received;
 - to make sure that `Ctrl+C` does not shut down the ROS context before the `finally` block is executed, `rcipy`'s default signal handler is disabled:


```
rcipy.init(args=args,
            signal_handler_options=SignalHandlerOptions.NO).
```

- Test the script in front of the yellow line:

Safety: the robot will move. Place it on the line, keep the track clear, and be ready to stop the script (`q`, `Esc` or `Ctrl+C`) or to lift the robot.

1. Bring up the TurtleBot3. Connect to the TurtleBot3 with SSH and, **on the TurtleBot3 RPi**, start the main node, which publishes the sensor topics and waits for velocity commands:

```
ros2 launch turtlebot3_bringup robot.launch.py
```

The output should end with:

```
[turtlebot3_ros-3] [INFO] [1790783254.414805329]
[turtlebot3_node]: Run!
[turtlebot3_ros-3] [INFO] [1790783254.461224488]
[diff_drive_controller]: Init Odometry
[turtlebot3_ros-3] [INFO] [1790783254.485064666]
[diff_drive_controller]: Run!
```

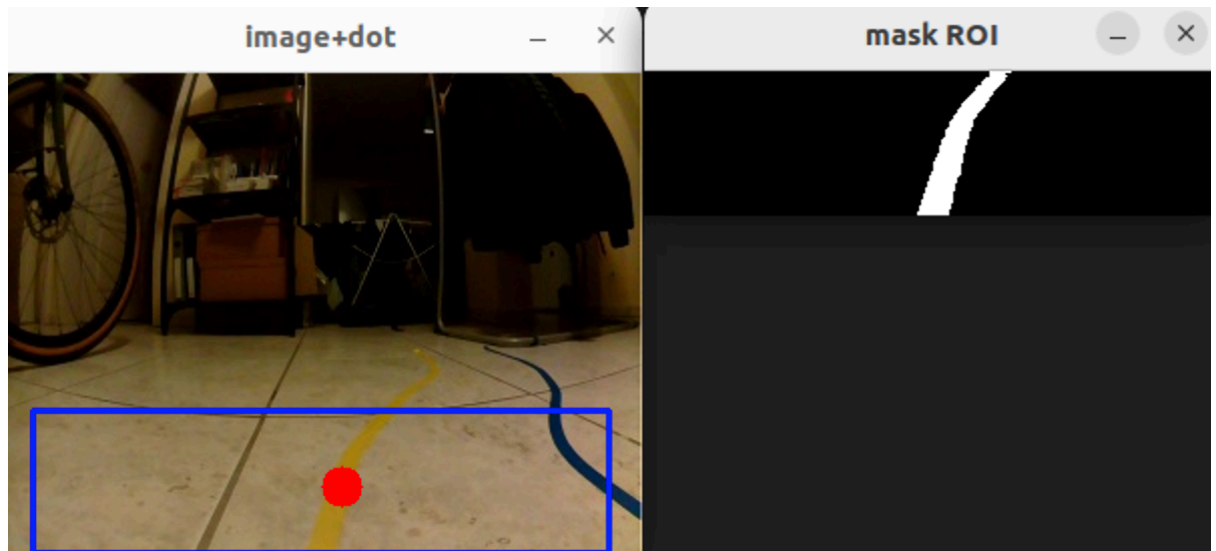
2. In a second terminal, connect to the TurtleBot3 with SSH again and, **on the TurtleBot3 RPi**, start the camera:

```
ros2 run v4l2_camera v4l2_camera_node --ros-args -p image_size:=
[320,240]
```

3. **On the PC**, run the script (after updating the compressed image topic if required):

```
python3 05follower.py
```

It should display the original image with a red dot at the centroid of **the yellow pixels inside the ROI** and, for debugging, the ROI mask (the ROI pixels where the color is detected):



And the robot should follow the line!

- Troubleshooting:
 - if the robot reacts too slowly to a change of direction of the line, or oscillates around it:
 - tune the **KP** gain (and possibly lower **LINEAR_VEL**),
 - adjust the ROI size and the orientation of the camera on the robot;
 - if the robot moves away from the line instead of going back to it:
 - check the sign of **err**,
 - check that the motors are not swapped: a positive angular velocity must turn the robot **counterclockwise** as seen from above (ROS convention). This can be tested with `ros2 run turtlebot3_teleop teleop_keyboard`.
- Going further: handling wheel velocity saturation
 - To turn, the robot drives one wheel faster than the other: each wheel velocity is $\text{LINEAR_VEL} \pm \text{angular.z} \times L/2$ (L = wheel separation, 0.160 m on the Burger). If **LINEAR_VEL** is close to the maximum velocity of the robot (0.22 m/s, i.e. the maximum velocity of the wheels), the outer wheel cannot go any faster: its velocity saturates and the robot turns less than commanded, or not at all.
 - Devise a strategy to handle this saturation, for example by reducing the linear velocity when the required angular velocity (i.e. the error) is large, so that turning always has priority over going forward.

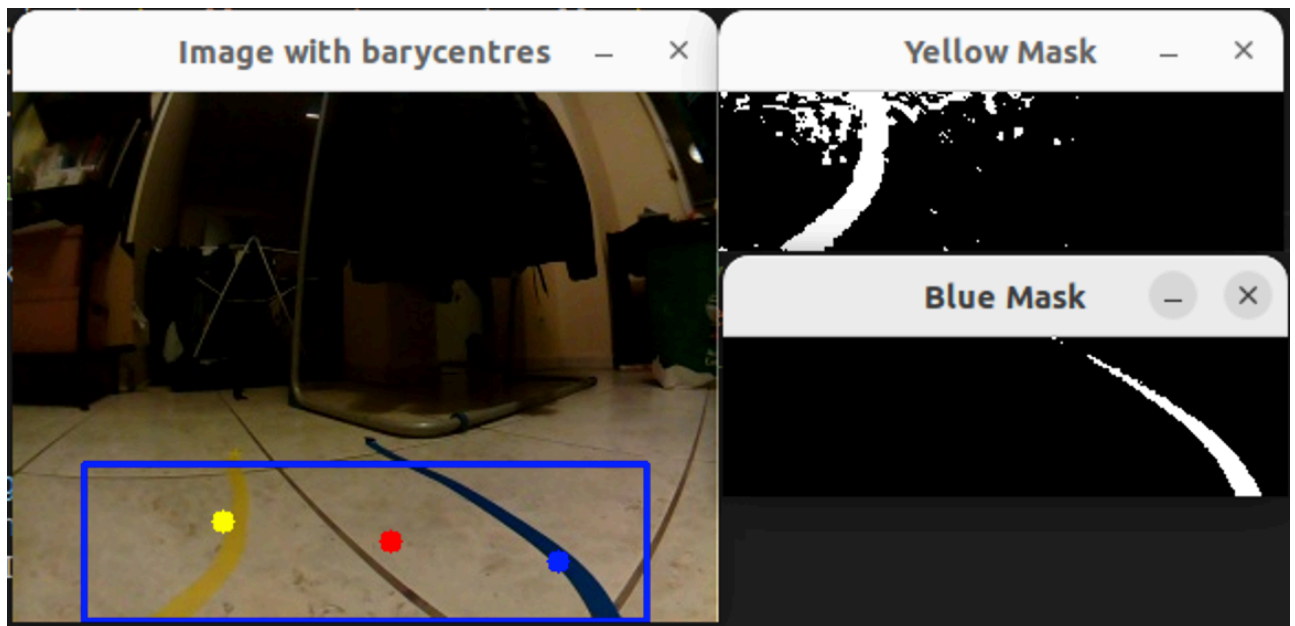
3. A track follower (two lines) — URMRC26

Track detection

The [autonomous_driving](#) project from Robotis provides camera calibration and lane detection, but it is somewhat cumbersome to set up. Here, we propose a simpler alternative, based on the previous line follower, that works as well and does not require any camera calibration.

- Principle:

- detect the centroid of the left line of the track (yellow, yellow dot in the screenshot below) and of the right line (blue, blue dot),
- compute the mean of the 2 centroids to get the center of the track (red dot).



- Open `src/track_detect.py`. Compared to `04follower_line_finder.py`, it defines two color ranges (`self.lower_yellow/self.upper_yellow` and `self.lower_blue/self.upper_blue`), computes one mask and one centroid per line, and computes the track center only when both lines are detected.
- Test:
 1. **On the TurtleBot3 RPi**, start the camera node.
 2. **On the PC**, run the script:

```
python3 track_detect.py
```

3. Adapt the HSV limits to the colors of your track lines and to the lighting conditions. Run the script again.

Track following

Once the position of the track with respect to the robot (i.e. with respect to the image center) is known, the robot can follow it.

Implementation

- By updating `track_detect.py` with the velocity control code of `05follower.py`, we obtain `track_follower.py`.
- Test it.
- Tune the parameters to improve the track following:

- adapt the **KP** gain,
- choose the best height of the bottom ROI, and the best width (full image width?) for track following.

Improving the track following

- **Strongly recommended:** handle the case where only one line of the track is visible (sharp turn, inaccurate following, ...):
 - add **elif** cases to **if cx_yellow is not None and cx_blue is not None:** (for example **elif cx_yellow is not None:**), where the position of the track in the image is estimated from the visible line plus or minus half the track width in pixels (**cx_yellow + half_width** if only the left yellow line is visible, **cx_blue - half_width** if only the right blue line is visible). **half_width** can be measured in the image when both lines are visible.
- Other ideas to improve the line detection and following:
 - set a threshold on the number of detected pixels before considering a line as detected. Currently, a line is considered detected as soon as a single pixel matches its color (**M_yellow['m00'] > 0**). This is done in the Robotis autorace project, in [detect_lane.py](#);
 - handle the wheel velocity saturation (see the end of [Step 5. Follow the line](#));
 - replace the proportional (P) controller with a proportional-derivative (PD) controller;
 - if several blobs of the same color are detected, keep only the biggest one (see **cv2.findContours()** and **cv2.contourArea()**);
 - ...

Appendix: embed the Python script in a ROS 2 package

Advanced users may want to launch the line/track follower node with the **ros2 run** command instead of calling Python directly. To do so, a ROS 2 package must be created around the Python script.

[Work in progress]